Document number: P1149R0
Project: Programming Language C++
Audience: Library Evolution Working group

Antony Polukhin, Yandex.Taxi Ltd, <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2018-10-01

Constexpr regex

I. Introduction and Motivation

Many existing programming languages compile/translate user-provided regular expressions before the actual program execution. This speeds up run times a lot, as the construction and optimization of the internal finite state machine from the regular expression is a time consuming operation.

Consider the following C++ example:

```
bool is_valid_mail(std::string_view mail) {
    static const std::regex mail_regex(R"((?:(?:[^<>()\[\].,;:\s@\"]+(?:\.[^<>()\[\].,;:\s@\"]+)*)|\".+\")@(?:(?:[^<>()\[\].,;:\s@\"]+\.)+[/

    return std::regex_match(
        std::cbegin(mail),
        std::cend(mail),
        mail_regex
    );
}
```

For C++17, the finite state machine for the above code is built at runtime at the first execution of the `is_valid_mail` function. What's worse, that time consuming operation is executed under the mutex.

With latest changes applied to the constant expressions it is now possible to significantly improve existing `std::basic_regex` by allowing it to construct a finite state machine at the compile time. We just need to mark `std::basic_regex` and functions that it use for building FSM with `constexpr`.

II. Impact on the Standard

This proposal is a pure library extension and it does not break the existing valid code and improves performance. It does not require any changes in the core language but relies on the upcoming core language features P0595 "std::is_constant_evaluated()" and P0784 "More constexpr containers".

III. std::locale and constexpr

libstdc++ and libc++ construct std::locale in the constructors of std::basic_regex. This prevents std::basic_regex constructors from being constexpr.

However, the only notes on std::locale construction are in [re.regex.locale]: "Returns the result of traits_inst.imbue(loc) where traits_inst is a (default-initialized) instance of the template type argument traits stored within the object.". This probably allows to have a optional<regex_traits> inside the std::basic_regex and initialize regex_traits (and the std::locale) on first usage.

So the lazy-initialization of std::locale in std::basic_regex allows to have constexpr std::basic_regex constructors.

IV. Proposed wording relative to N4741

All the additions to the Standard are marked with underlined green.

Green lines are notes for the **editor** that must not be treated as part of the wording.

1. Modifications to "Class template regex_traits" [re.regex]

```
namespace std {
  template<class charT>
  struct regex_traits {
    using char_type = charT;
    using string_type = basic_string<char_type>;
    using locale_type = locale;
    using char_class_type = bitmask_type;

    regex_traits();
    static constexpr size_t length(const char_type* p);
    charT translate(charT c) const;
    charT translate_nocase(charT c) const;
    template<class ForwardIterator>
      string_type transform(ForwardIterator first, ForwardIterator last) const;
    template<class ForwardIterator>
      string_type transform_primary(
        ForwardIterator first, ForwardIterator last) const;
    template<class ForwardIterator>
      string_type lookup_collatename(
        ForwardIterator first, ForwardIterator last) const;
    template<class ForwardIterator>
      char_class_type lookup_classname(
        ForwardIterator first, ForwardIterator last, bool icase = false) const;
    bool isctype(charT c, char_class_type f) const;
    int value(charT ch, int radix) const;
    locale_type imbue(locale_type l);
    locale_type getloc() const;
```

```
  };
}
```

All the functions marked with `constexpr` in previous paragraph of this document must be accordingly marked with `constexpr` in detailed description of `regex_traits` functions.

2. Modifications to "Class template basic_regex" [re.traits]

Add the following paragraph after the second paragraph:

Internal finite state machine construction is a constant expression if arguments of the `basic_regex` constructor are constant expressions.

Adjust the synopsis after the Table 124 "Character class names and corresponding ctype masks":

```
    <...>

    // 31.8.1, construct/copy/destroy
    constexpr basic_regex();
    explicit constexpr basic_regex(const charT* p, flag_type f = regex_constants::ECMAScript);
    constexpr basic_regex(const charT* p, size_t len, flag_type f = regex_constants::ECMAScript);
    constexpr basic_regex(const basic_regex&);
    constexpr basic_regex(basic_regex&&) noexcept;
    template<class ST, class SA>
    explicit constexpr basic_regex(const basic_string<charT, ST, SA>& p,
        flag_type f = regex_constants::ECMAScript);
    template<class ForwardIterator>
    constexpr basic_regex(ForwardIterator first, ForwardIterator last,
        flag_type f = regex_constants::ECMAScript);
    constexpr basic_regex(initializer_list<charT>, flag_type = regex_constants::ECMAScript);

    constexpr ~basic_regex();

    constexpr basic_regex& operator=(const basic_regex&);
    constexpr basic_regex& operator=(basic_regex&&) noexcept;
    constexpr basic_regex& operator=(const charT* ptr);
    constexpr basic_regex& operator=(initializer_list<charT> il);
    template<class ST, class SA>
    constexpr basic_regex& operator=(const basic_string<charT, ST, SA>& p);

    // 31.8.2, assign
    constexpr basic_regex& assign(const basic_regex& that);
    constexpr basic_regex& assign(basic_regex&& that) noexcept;
    constexpr basic_regex& assign(const charT* ptr, flag_type f = regex_constants::ECMAScript);
    constexpr basic_regex& assign(const charT* p, size_t len, flag_type f);
    template<class string_traits, class A>
    constexpr basic_regex& assign(const basic_string<charT, string_traits, A>& s,
        flag_type f = regex_constants::ECMAScript);
    template<class InputIterator>
    constexpr basic_regex& assign(InputIterator first, InputIterator last,
        flag_type f = regex_constants::ECMAScript);
    constexpr basic_regex& assign(initializer_list<charT>,
        flag_type = regex_constants::ECMAScript);

    // 31.8.3, const operations
    constexpr unsigned mark_count() const;
    constexpr flag_type flags() const;

    // 31.8.4, locale
    locale_type imbue(locale_type l);
    locale_type getloc() const;

    // 31.8.5, swap
    constexpr void swap(basic_regex&);
  };

  template<class ForwardIterator>
  basic_regex(ForwardIterator, ForwardIterator, regex_constants::syntax_option_type = regex_constants::ECMAScript)
    -> basic_regex<typename iterator_traits<ForwardIterator>::value_type>;
}
```

All the functions marked with `constexpr` in previous paragraph of this document must be accordingly marked with `constexpr` in [re.traits.construct], [re.regex.assign], [re.regex.operations] and [re.regex.swap].

3. Feature-testing macro

Add a row into the "Standard library feature-test macros" table [support.limits.general]:

`__cpp_lib_constexpr_regex`	201811	<regex>